UNIT I

INTRODUCTION: Origins and challenges of NLP – Language Modeling: Grammar-based LM, Statistical LM – Regular Expressions, Finite-State Automata – English Morphology, Transducers for lexicon and rules, Tokenization, Detecting and Correcting Spelling Errors, Minimum Edit Distance.

Natural Language Processing (NLP) is a branch of **Artificial Intelligence (AI)** that focuses on enabling computers to understand, interpret, and generate human language, both spoken and written. For students, understanding its journey and obstacles is key to grasping the field.

Origins of NLP: A Historical Overview

The development of NLP can be broken down into three main approaches:

1. The Dawn: Rule-Based (1950s - 1980s)

The earliest attempts relied on manually coding grammatical rules and vocabulary into the system.

- **1950: The Turing Test:** Alan Turing proposed his famous test, challenging machines to exhibit human-like intelligence in conversation, laying the **theoretical foundation** for conversational AI.
- **1954: Georgetown-IBM Experiment:** This was one of the first major demonstrations of **Machine Translation (MT)**, successfully translating a limited set of Russian sentences into English using simple rules.
- **Mid-1960s: ELIZA:** Developed by Joseph Weizenbaum, ELIZA was a key early chatbot that simulated a Rogerian psychotherapist. It used simple **pattern matching** and template-based responses (e.g., if you say "My head hurts," it might reply, "Why do you say your head hurts?"). Crucially, it did **not truly understand** the language.
- **1970s: SHRDLU:** Terry Winograd's program demonstrated sophisticated natural language interaction within a very **restricted world** (a blocks world), highlighting the potential and limitations of rule-based systems.

2. The Statistical Revolution (1980s - 2000s)

Researchers realized that language was too complex and unpredictable for hand-coded rules. The focus shifted to using **statistical methods** and **Machine Learning (ML)** to learn patterns from large amounts of text data.

- **1980s - 1990s: Statistical Models:** Techniques like **Hidden Markov Models (HMMs)** and **Probabilistic Context-Free Grammars (PCFGs)** became popular for tasks like Part-of-Speech (POS) tagging and speech recognition. The systems calculated the **probability** of a word or sequence occurring.
- **2006: Google Translate (SMT):** The initial version of Google Translate used **Statistical Machine Translation (SMT)**, leveraging massive amounts of parallel text (text translated by humans) to achieve significantly better accuracy than previous rule-based translators.

3. The Deep Learning Era (2010s - Present)

The rise of neural networks and abundant computational power (GPUs/TPUs) fundamentally transformed NLP.

- **2013: Word Embeddings (Word2Vec, GloVe):** These techniques allowed words to be represented as dense numerical vectors, capturing their **semantic relationships**. For

instance, the distance between the vector for "King" and "Queen" is similar to the distance between "Man" and "Woman."

- **2015: Neural Machine Translation (NMT):** Google Translate switched to an NMT system using recurrent neural networks (RNNs) and Long Short-Term Memory (LSTMs), leading to more fluent and context-aware translations.
- **2017: The Transformer Architecture:** Google's paper "Attention Is All You Need" introduced the **Transformer architecture**. This model uses **self-attention** to weigh the importance of different words in a sequence, enabling it to model long-range dependencies far better than RNNs. This is the **foundation of modern NLP**.
- **2018 - Present: Large Language Models (LLMs):** Transformer-based models like **BERT** (Bidirectional Encoder Representations from Transformers) and the **GPT series** (Generative Pre-trained Transformer) are pre-trained on massive text datasets and have set new benchmarks for text generation, comprehension, and reasoning.

Major Challenges in NLP

Despite recent breakthroughs, the complexities of human language present several significant challenges for NLP systems.

1. Linguistic Challenges

- **Ambiguity and Polysemy:** The most fundamental challenge is that a single word or phrase can have **multiple meanings**.
    - **Lexical Ambiguity:** A single word having multiple meanings (e.g., "The **bank** is slippery" vs. "I went to the **bank**").
    - **Syntactic Ambiguity:** A sentence having multiple valid grammatical structures (e.g., "I saw the man with the telescope").
- **Contextual Understanding:** The meaning of an utterance often relies on the broader context, shared knowledge, or real-world common-sense.
    - **Coreference Resolution:** Identifying when different expressions refer to the same entity (e.g., "John is a doctor. **He** went to the hospital.").
    - **Sarcasm, Irony, and Figurative Language:** These expressions defy literal meaning and are extremely difficult for models to reliably interpret.
- **Pragmatics and Discourse:** Understanding the intent behind the words (Pragmatics) and how meaning flows across multiple sentences (Discourse) remains complex. For example, understanding that "Can you pass the salt?" is a request, not a question about ability.
- **Variability and Noise:** Real-world language includes typos, grammatical errors, slang, abbreviations (especially in social media), and regional dialects, which can confuse NLP models trained on clean data.
- **Multilingualism and Low-Resource Languages:** Many languages lack the vast datasets, computational resources, and established research tools available for languages like English. Building high-quality, universal systems remains a significant hurdle.

2. Technical and Data Challenges

- **Data Quality and Bias:** NLP models are only as good as their training data. If the data contains societal biases (e.g., racial, gender), the model will learn and **perpetuate those biases**, leading to unfair or discriminatory outputs (e.g., the case of the Tay chatbot).

- **Lack of Annotated Data:** For many specialized tasks (e.g., named entity recognition in medical documents), large datasets need to be manually labeled (**annotated**), which is time-consuming and expensive.
- **Computational Intensity:** Training and running the most advanced LLMs (like GPT-4) requires **massive computational resources** (GPUs/TPUs) and immense energy, making them inaccessible to many researchers and organizations.
- **Explainability (XAI):** Advanced deep learning models are often considered **"black boxes,"** meaning it is difficult to understand *why* they produced a specific output. This lack of transparency is a major concern in sensitive applications like medicine or law.
- **Hallucination and Factuality:** Generative models can produce text that sounds fluent but is factually incorrect or nonsensical (**hallucination**). Ensuring the model's output is grounded in truth is an ongoing research area.

Language Modeling: Grammar-Based Approach

A **Grammar-Based Language Model** is a system that uses **formal, explicit grammar rules** to determine if a sentence is correctly formed (syntactically correct) and to assign a probability to that sentence. It focuses on the *structure* of language, not just the sequence of words.

Language Modeling (LM) is a core task in Natural Language Processing (NLP) where the goal is to **assign a probability** to a sequence of words, $P(w_1, w_2, \ldots, w_n)$, or to predict the next word in a sequence, $P(w_n | w_1, \ldots, w_{\{n-1\}})$.

Grammar-based Language Models (LMs) belong to the historical **Rule-Based** approach in NLP, in contrast to modern statistical and neural models. They focus on defining language structure through formal, explicit rules.

A grammar-based approach in NLP uses formal rules (like Context-Free Grammars) to define sentence structure, enabling machines to parse text, understand syntax, and generate grammatically correct language, acting as a foundation for language modeling before modern statistical methods dominated, but still relevant in hybrid systems for syntax enforcement and understanding structural relationships, moving beyond simple word prediction to structural comprehension.

How it Works

- Grammar Rules: Define how words form phrases (Noun Phrases, Verb Phrases) and sentences (e.g., S -> NP VP).
- Parsing: An input sentence is analyzed (parsed) to build a parse tree, showing its hierarchical structure.
- Rule Application: These rules guide text generation or interpretation, ensuring output adheres to the language's syntax.

Key Concepts

- **Grammar:** A formal set of rules describing syntax (e.g., CFGs, Dependency Grammars).
- **Syntax:** Word arrangement for well-formed sentences.
- **Parsing:** Analyzing a sentence to find its structure using grammar rules.

What is a Language Model in Natural Language Processing?

A language model in natural language processing (NLP) is a statistical or machine learning model that is used to predict the next word in a sequence given the previous words. Language models play a crucial role in various NLP tasks such as machine translation, speech recognition, text generation, and sentiment analysis. They analyze and understand the structure and use of human language, enabling machines to process and generate text that is contextually appropriate and coherent.

Language models can be broadly categorized into two types:

- Pure Statistical Methods
- Neural Models

a) Pure Statistical Methods

Pure statistical methods form the basis of traditional language models. These methods rely on the statistical properties of language to predict the next word in a sentence, given the previous words. They include n-grams, exponential models, and skip-gram models.

1. N-grams

An n-gram is a sequence of n items from a sample of text or speech, such as phonemes, syllables, letters, words, or base pairs. N-gram models use the frequency of these sequences in a training corpus to predict the likelihood of word sequences. For example, a bigram (2-gram) model predicts the next word based on the previous word, while a trigram (3-gram) model uses the two preceding words.

N-gram models are simple, easy to implement, and computationally efficient, making them suitable for applications with limited computational resources. However, they have significant limitations. They struggle with capturing long-range dependencies due to their limited context window. As n increases, the number of possible n-grams grows exponentially, leading to sparsity issues where many sequences are never observed in the training data. This sparsity makes it difficult to accurately estimate the probabilities of less common sequences.

2. Exponential Models

Exponential models, such as the Maximum Entropy model, are more flexible and powerful than n-gram models. They predict the probability of a word based on a wide range of features, including not only the previous words but also other contextual information. These models assign weights to different features and combine them using an exponential function to estimate probabilities.

Maximum Entropy Models

Maximum Entropy (MaxEnt) models, also known as logistic regression in the context of classification, are used to estimate the probabilities of different outcomes based on a set of features. In the context of language modeling, MaxEnt models use features such as the presence of certain words, part-of-speech tags, and syntactic patterns to predict the next word. The model parameters are learned by maximizing the likelihood of the observed data under the model.

MaxEnt models are more flexible than n-gram models because they can incorporate a wider range of features. However, they are also more complex and computationally intensive

to train. Like n-gram models, MaxEnt models still struggle with long-range dependencies because they rely on fixed-length context windows.

3. Skip-gram Models

Skip-gram models are a type of statistical method used primarily in word embedding techniques. They predict the context words (surrounding words) given a target word within a certain window size. Skip-gram models, particularly those used in Word2Vec, are effective for capturing the semantic relationships between words by optimizing the likelihood of context words appearing around a target word.

Word2Vec and Skip-gram

Word2Vec, developed by Google, includes two main architectures: skip-gram and continuous bag-of-words (CBOW). The skip-gram model predicts the context words given a target word, while the CBOW model predicts the target word given the context words. Both models are trained using neural networks, but they are conceptually simple and computationally efficient.

b)Neural Models

Neural models have revolutionized the field of NLP by leveraging deep learning techniques to create more sophisticated and accurate language models. These models include Recurrent Neural Networks (RNNs), Transformer-based models, and large language models.

1. Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a type of neural network designed for sequential data, making them well-suited for language modeling. RNNs maintain a hidden state that captures information about previous inputs, allowing them to consider the context of words in a sequence.

LSTMs and GRUs are advanced RNN variants that address the vanishing gradient problem, enabling the capture of long-range dependencies in text. LSTMs use a gating mechanism to control the flow of information, while GRUs simplify the gating mechanism, making them faster to train.

2. Transformer-based Models

The Transformer model, introduced by Vaswani et al. in 2017, has revolutionized NLP. Unlike RNNs, which process data sequentially, the Transformer model processes the entire input simultaneously, making it more efficient for parallel computation.

The key components of the Transformer architecture are:

- **Self-Attention Mechanism**: This mechanism allows the model to weigh the importance of different words in a sequence, capturing dependencies regardless of their distance in the text. Each word's representation is updated based on its relationship with all other words in the sequence.
- **Encoder-Decoder Structure**: The Transformer consists of an encoder and a decoder. The encoder processes the input sequence and generates a set of hidden representations. The decoder takes these representations and generates the output sequence.
- **Positional Encoding**: Since Transformers do not process the input sequentially, they use positional encoding to retain information about the order of words in a sequence. This encoding adds positional information to the input embeddings, allowing the model to consider the order of words.
- Some of the transformers based models are - BERT, GPT-3, T5 and more.

3. Large Language Models (LLMs)

Large language models have pushed the boundaries of what is possible in NLP. These models are characterized by their vast size, often comprising billions of parameters, and their ability to perform a wide range of tasks with minimal fine-tuning.

Training large language models involves feeding them vast amounts of text data and optimizing their parameters using powerful computational resources. The training process typically includes multiple stages, such as unsupervised pre-training on large corpora followed by supervised fine-tuning on specific tasks.

While large language models offer remarkable performance, they also pose significant challenges. Training these models requires substantial computational resources and energy, raising concerns about their environmental impact. Additionally, the models' size and complexity can make them difficult to interpret and control, leading to potential ethical and bias issues.

Strengths & Limitations

- **Strength:** Explicitly models linguistic structure, offering deep grammatical understanding.
- **Limitation:** Handwritten grammars are complex; grammatical inference is hard; struggles with exceptions and ambiguity.

Evolution & Modern Context

- **Shift:** Moved from purely rule-based systems towards statistical (N-gram, neural) models that learn from data.
- **Hybrid Models:** Modern systems sometimes blend statistical power with grammar rules (e.g., Lexicalized Grammars) for better structure.
- **Relevance:** Still foundational for tasks needing structural precision, complementing data-driven methods.

Statistical LM – Regular Expressions

In Natural Language Processing (NLP), Regular Expressions (Regex) are fundamental tools for defining search patterns to find, manipulate, and extract structured data from unstructured text, acting as crucial pre-processing steps for tasks like cleaning (removing punctuation), tokenization (splitting words), and complex information extraction (emails, dates), often working alongside statistical models by providing rule-based features or handling simpler text tasks, even for complex Deep Learning Models.

They define patterns (e.g., \w+ for words, \d{3}-\d{3}-\d{4} for phone numbers) to identify and extract specific text segments, complementing statistical approaches by efficiently managing text structure and patterns.

While Regular Expressions (Regex) are a rule-based, symbolic approach to string manipulation, they are an absolutely essential tool in the preprocessing stage of virtually all Statistical Language Models (SLMs) in Natural Language Processing (NLP).

How Regex Works in NLP

- **Pattern Matching:** Regex uses a sequence of characters (literals and metacharacters like `*`, `+`, `.`) to describe a search pattern in text, matching specific sequences or structures.
- **Text Cleaning & Normalization:** Removing unwanted characters, standardizing formats (e.g., all lowercase), or removing HTML tags.
- **Tokenization:** Splitting text into meaningful units (words, sentences) using patterns like `\w+` for words or `.` for sentence endings.
- **Feature Engineering:** Extracting features for statistical models, such as identifying all-caps words or specific number formats.
- **Information Extraction:** Pulling out structured data like emails, dates, phone numbers, or URLs.

Regex vs. Statistical/Neural Models (LLMs)

- **Regex (Rule-Based):** Excellent for precise, deterministic pattern matching (e.g., "find all 10-digit numbers"). Fast and efficient for specific tasks.
- **Statistical/Neural Models (e.g., N-grams, Transformers/LLMs):** Learn complex, probabilistic patterns from data, handling ambiguity and context better (e.g., understanding sarcasm, generating fluent text).
- **Hybrid Approach (Common):** Regex handles the simple, predictable parts (cleaning, exact entity matching), while complex models handle the nuanced, contextual understanding, creating powerful, efficient NLP pipelines.

Examples in NLP

- **Tokenization:** `\b[A-Za-z]+\b` matches whole words.
- **Email Matching:** `[\w.-]+@[\w.-]+\.\w+` finds typical email formats.
- **Phone Numbers:** `\b\d{3}-\d{3}-\d{4}\b` finds XXX-XXX-XXXX patterns.

Finite-State Automata – English Morphology

Finite-State Automata (FSA) and especially **Finite-State Transducers (FSTs)** are fundamental in English Morphology in NLP, modeling word structures by breaking them into stems and affixes (like `cat` + `s` for "cats"). FSTs map surface words to underlying meanings (lexical forms) and vice-versa, handling regular and irregular inflections (e.g., 'walk' vs. 'go') using cascades of automata for lexicon, rules, and spelling changes, providing powerful, compact, and visual ways to process word formation in NLP.

How they work in English Morphology:

- **Morphological Analysis**: FSTs parse words (e.g., "running") into morphemes (root `run` + suffix `ing`).
- **Lexical vs. Surface Forms**: One tape (lexical) holds abstract parts (stem+features), the other (surface) holds the actual word.
- **Transduction**: An FST translates between these tapes (e.g., `run` + `-ing` -> `running` or `running` -> `run` + `-ing`).
- **Cascades**: Multiple FSTs work in sequence: one for the lexicon, one for morphotactics (rules), and one for spelling changes (e.g., dropping 'e' before adding '-ing').

Key Concepts:

- **FSA**: Recognizes if a string is a valid word in a language.
- **FST**: Maps input string pairs, crucial for understanding word structure (e.g., stem + affix).
- **Morphemes**: Smallest meaningful parts (stems, prefixes, suffixes).
- **Regular vs. Irregular**: Handles common patterns (walk-ed) and exceptions (go-went).

Advantages:

- **Compact & Efficient**: Compact representation of lexical rules and phenomena.
- **Linguistic Intuition**: Maps well to linguistic descriptions and is visually understandable.
- **Dual Functionality**: Acts as recognizer, generator, or translator.

Limitations:

- **Manual Design**: Requires significant linguistic expertise.
- **Scalability**: Can become complex with many rules.
- **Generalization**: Struggles with ambiguity or unseen data compared to neural models.

Transducers for lexicon and rules

In NLP, **Finite State Transducers (FSTs)** are powerful tools that model the relationship between a word's surface form (like "cats") and its underlying components (stem "cat," noun, plural), efficiently handling both the **lexicon** (word parts) and **rules** (morphological/spelling changes) through a single machine. They work like two-tape automatons, mapping input strings to output strings, making them ideal for morphological analysis (parsing words), generation, spelling correction, and even simple syntactic tasks by encoding lexical entries and linguistic rules (like suffix addition or letter changes) into states and transitions.

How FSTs Combine Lexicon & Rules:

1. **Lexicon (Word Parts):**

- The lexicon is represented by basic morphemes (stems, prefixes, suffixes).
- An FST links a surface form like "unhappily" to its lexical parts: `un + happy + ly`.

2. **Rules (Morphotactic & Orthographic):**

- **Morphotactic Rules:** Define how morphemes combine (e.g., a plural `-s` follows a noun stem).
- **Orthographic Rules:** Handle spelling changes (e.g., changing 'y' to 'i' before adding '-es' for plural 'babies').
- These rules are encoded directly into the FST's transitions (e.g., a transition for the 's' in "cats" outputs 's' but might also signal a plural feature).

Key Applications in NLP:

- **Morphological Parsing:** Decomposing words like "running" into `run + V + PRES`, as shown in GeeksforGeeks.
- **Word Generation:** Creating inflected forms (e.g., "play" -> "played").
- **Spelling Correction:** Identifying and fixing typos or applying rules.
- **Tokenization:** Segmenting text into meaningful units.

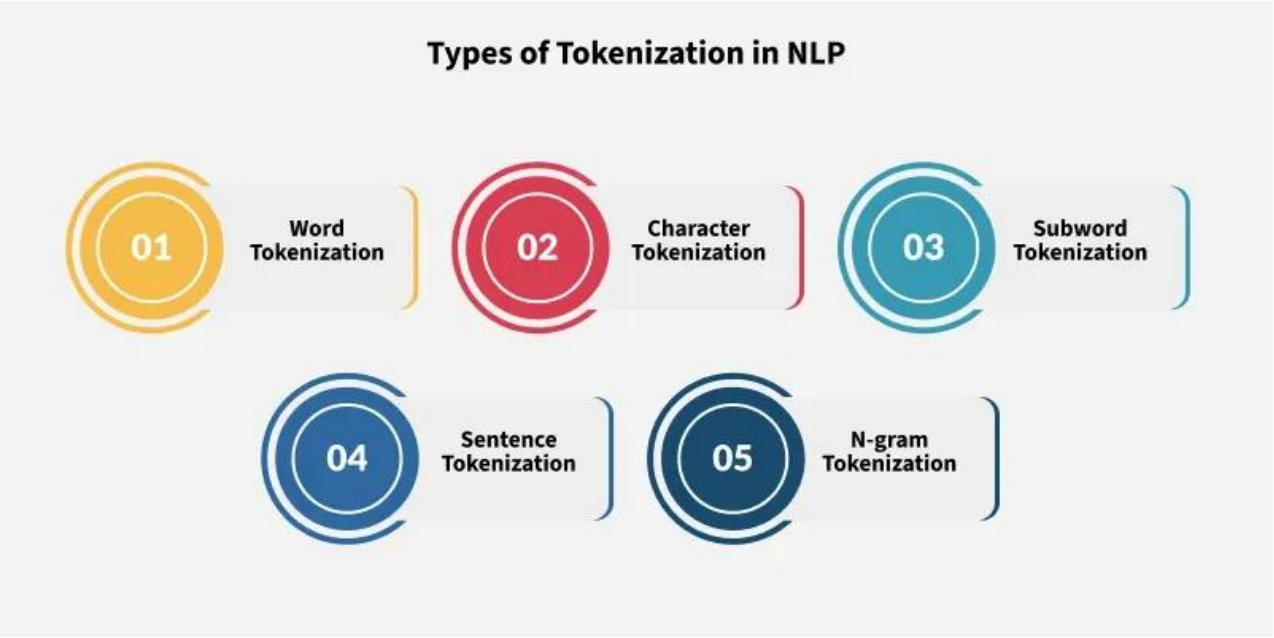
Tokenization

Tokenization is a fundamental step in Natural Language Processing (NLP). It involves dividing a Textual input into smaller units known as tokens. These tokens can be in the form of words, characters, sub-words, or sentences. It helps in improving interpretability of text by different models. Let's understand How Tokenization Works.

Natural Language Processing (NLP) is a subfield of Artificial Intelligence, information engineering, and human-computer interaction. It focuses on how to process and analyze large amounts of natural language data efficiently. It is difficult to perform as the process of reading and understanding languages is far more complex than it seems at first glance.

- Tokenization is a foundation step in NLP pipeline that shapes the entire workflow.
- Involves dividing a string or text into a list of smaller units known as tokens.
- Uses a tokenizer to segment unstructured data and natural language text into distinct chunks of information, treating them as different elements.

- Tokens: Words or Sub-words in the context of natural language processing. Example: A word is a token in a sentence, A character is a token in a word, etc.
- Application: Multiple NLP tasks, text processing, language modelling, and machine translation.



Tokenization can be classified into several types based on how the text is segmented. Here are some types of tokenization:

1. Word Tokenization

Word tokenization is the most commonly used method where text is divided into individual words. It works well for languages with clear word boundaries, like English.

For example, "Machine learning is fascinating" becomes:

Input before tokenization*: ["Machine Learning is fascinating"]*
Output when tokenized by words*: ["Machine", "learning", "is", "fascinating"]*

2. Character Tokenization

In Character Tokenization, the textual data is split and converted to a sequence of individual characters. This is beneficial for tasks that require a detailed analysis, such as spelling correction or for tasks with unclear boundaries. It can also be useful for modelling character-level language.

Example

Input before tokenization*: ["You are helpful"]*
Output when tokenized by characters*: ["Y", "o", "u", " ", "a", "r", "e", " ", "h", "e", "l", "p", "f", "u", "l"]*

3. Sub-word Tokenization

This strikes a balance between word and character tokenization by breaking down text into units that are larger than a single character but smaller than a full word. This is useful when dealing with morphologically rich languages or rare words.

Example

["Time", "table"]
["Rain", "coat"]
["Grace", "fully"]
["Run", "way"]

Sub-word tokenization helps to handle out-of-vocabulary words in NLP tasks and for languages that form words by combining smaller units.

4. Sentence Tokenization

Sentence tokenization is also a common technique used to make a division of paragraphs or large set of sentences into separated sentences as tokens. This is useful for tasks requiring individual sentence analysis or processing.

***Ex: Input before tokenization**: ["Artificial Intelligence is an emerging technology. Machine learning is fascinating. Computer Vision handles images. "]*

***Output when tokenized by sentences** ["Artificial Intelligence is an emerging technology.", "Machine learning is fascinating.", "Computer Vision handles images."]*

5. N-gram Tokenization

N-gram tokenization splits words into fixed-sized chunks (size = n) of data.

***Input before tokenization**: ["Machine learning is powerful"]*

***Output when tokenized by bigrams**: [('Machine', 'learning'), ('learning', 'is'), ('is', 'powerful')]*

Need of Tokenization

Tokenization is an essential step in text processing and natural language processing (NLP) for several reasons. Some of these are listed below:

- **Effective Text Processing:** Reduces the size of raw text, resulting in easy and efficient statistical and computational analysis.
- **Feature extraction:** Text data can be represented numerically for algorithmic comprehension by using tokens as features in ML models.
- **Information Retrieval:** Tokenization is essential for indexing and searching in systems that store and retrieve information efficiently based on words or phrases.
- **Text Analysis:** Used in sentiment analysis and named entity recognition, to determine the function and context of individual words in a sentence.

- **Vocabulary Management:** Generates a list of distinct tokens, Helps manage a corpus's vocabulary.
- **Task-Specific Adaptation:** Adapts to need of particular NLP task, Good for summarization and machine translation.

Limitations of Tokenization

- Unable to capture the meaning of the sentence hence, results in ambiguity.
- Chinese, Japanese, Arabic, lack distinct spaces between words. Hence, absence of clear boundaries that complicates the process of tokenization.
- Tough to decide how to tokenize text that may include more than one word, for example email address, URLs and special symbols

Detecting and Correcting Spelling Errors

Detecting and correcting spelling errors in NLP involves using techniques like dictionary lookups, n-grams, and edit distance algorithms to identify misspelled words and suggest corrections. These methods compare words against a dictionary or a language model to find mismatches, and then generate likely corrections by considering operations like insertions, deletions, or substitutions, with more advanced methods using machine learning and dynamic programming.

Detection and correction techniques

- **Dictionary lookup:**
  The most basic method is to check if a word exists in a predefined dictionary, often implemented with a hash table for fast lookups. If a word is not found, it is flagged as a potential error.
- **N-gram models:**
  These techniques analyze sequences of characters (n-grams) to find words with non-existent or infrequent n-grams, which are often misspellings.
- **Edit distance:**
  This calculates the minimum number of single-character edits (insertions, deletions, or substitutions) required to change one word into another. A low edit distance between a misspelled word and a dictionary word suggests a high probability of a correct match.
    - **Dynamic Programming (DP):** A more efficient way to calculate edit distance, which breaks the problem into smaller subproblems to find the optimal solution.
- **Machine Learning and Deep Learning:**
  More sophisticated models, such as those based on Long Short-Term Memory (LSTM) networks, can learn complex patterns in language to perform both detection and correction, often achieving higher accuracy.
- **Hybrid approaches:**
  Combining rule-based methods with deep learning can improve a system's ability to spot complex errors, as noted in this document from The Science and Information (SAI) Organization.

Tools and implementation

- **Python libraries:**
  Python has several libraries that can be used for this task.
  - **SymSpell:** A fast and accurate library that uses precomputed dictionaries and phonetic similarity to suggest corrections, as described in this GeeksforGeeks article.
  - **TextBlob:** A user-friendly library for processing textual data that includes spelling correction capabilities.
  - **Norvig, BK-tree, and LinSpell:** Other libraries mentioned in this GeeksforGeeks article as tools for spelling correction.
- **Process:**
  A typical process involves tokenizing text, using a library to identify and correct misspelled words, and then rejoining the words into a corrected sentence, as demonstrated in this YouTube video.

Challenges and considerations

- **Contextual errors:**
  Standard spell checkers struggle with "real-word" errors, where a word is spelled correctly but is used in the wrong context (e.g., "their" vs. "there").

- **Proper nouns:**
  Detecting errors in proper nouns can be difficult as they may not be in the standard dictionary.

- **Homophones:**
  The system must be able to distinguish between words that sound alike but have different meanings (e.g., "to," "too," and "two").

- **Continuous improvement:**
  Language evolves, so models need to be continuously retrained and updated with new data to maintain accuracy, according to this SAI document.

Minimum Edit Distance

Minimum Edit Distance, often referred to as Levenshtein distance, is a metric used in Natural Language Processing (NLP) to quantify the similarity or dissimilarity between two strings. It is defined as the minimum number of single-character editing operations (insertions, deletions, or substitutions) required to transform one string into another.

Key Concepts:

- **Operations:** The fundamental operations involved are:
  - **Insertion:** Adding a character to a string.
  - **Deletion:** Removing a character from a string.
  - **Substitution:** Replacing one character with another.

- **Cost:** Each operation typically carries a cost, most commonly a cost of 1 for each insertion, deletion, or substitution in the standard Levenshtein distance. Weighted versions can assign different costs based on the likelihood or type of operation.

- **Dynamic Programming:** The minimum edit distance is typically computed using a dynamic programming approach. This involves constructing a matrix where each cell D(i,j) represents the minimum edit distance between the prefix of the first string of length i and the prefix of the second string of length j. The matrix is filled iteratively, building solutions from sub-problems.

Applications in NLP:

- **Spell Correction:** Identifying potential correct spellings for misspelled words by finding words in a dictionary with a small edit distance.
- **Plagiarism Detection:** Measuring the similarity between documents or text segments to detect potential plagiarism.
- **Machine Translation and Speech Recognition Evaluation:** Assessing the quality of translations or transcriptions by comparing them to reference texts.
- **Named Entity Extraction and Coreference Resolution:** Identifying and linking mentions of entities within text.
- **Computational Biology:** Analyzing DNA sequences to understand genetic relationships.

Example:

To transform "kitten" into "sitting":

- **s**: itten (substitution of 'k' with 's')
- sittin (substitution of 'e' with 'i')
- sitting (insertion of 'g')
  The minimum edit distance is 3.